

1. Introdução

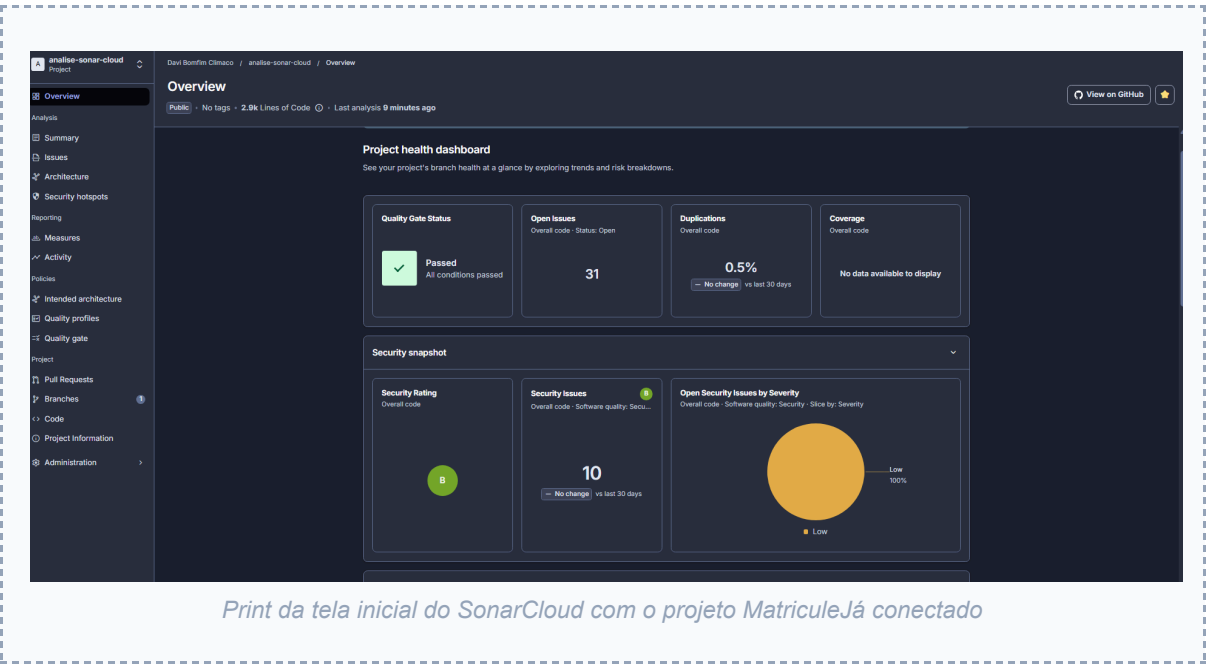
O MatriculeJá é um sistema de gestão de vagas em creches públicas municipais, desenvolvido como AEP da disciplina ESOFT. Para a etapa de Manutenção de Software, rodamos o SonarCloud no repositório do projeto com o objetivo de identificar pontos de melhoria no código-fonte e garantir que o sistema possa ser mantido e evoluído com mais facilidade no futuro.

O SonarCloud é uma ferramenta de análise estática que inspeciona automaticamente o código em busca de bugs, vulnerabilidades, code smells e problemas de cobertura de testes. A análise foi feita diretamente no repositório do GitHub, sem nenhuma configuração adicional.

2. Ferramenta Utilizada — SonarCloud

O SonarCloud foi escolhido por ser gratuito para projetos públicos, por se integrar diretamente com o GitHub e por oferecer uma interface visual clara com os resultados organizados por categoria. A análise cobre quatro dimensões principais:

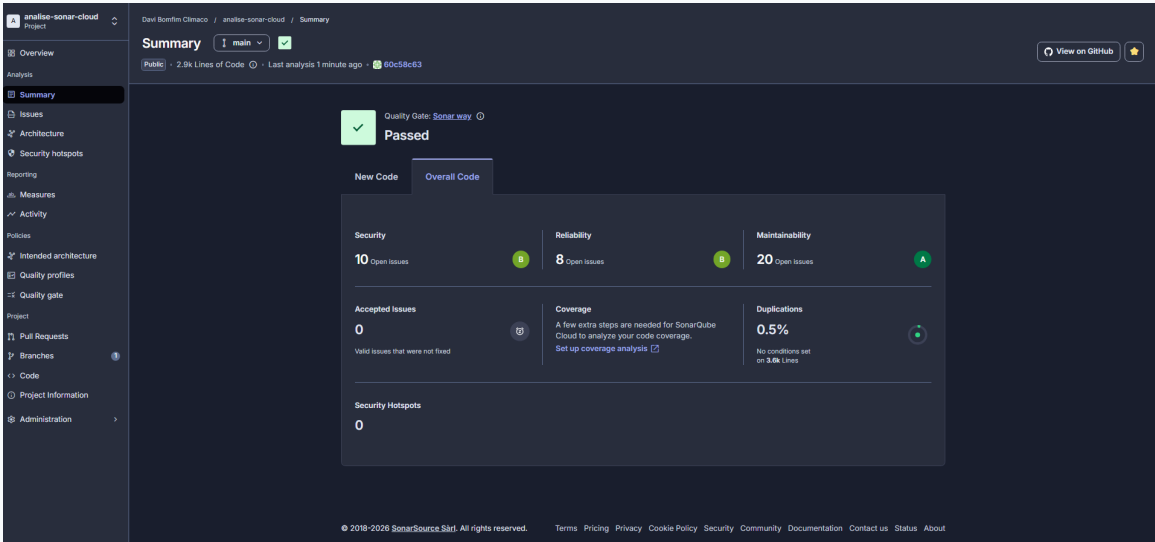
Categoria	O que analisa
Bugs	Erros no código que podem causar comportamento inesperado em produção
Vulnerabilities	Falhas de segurança que podem expor o sistema a ataques
Code Smells	Más práticas que dificultam a leitura e manutenção do código
Coverage	Percentual do código coberto por testes automatizados



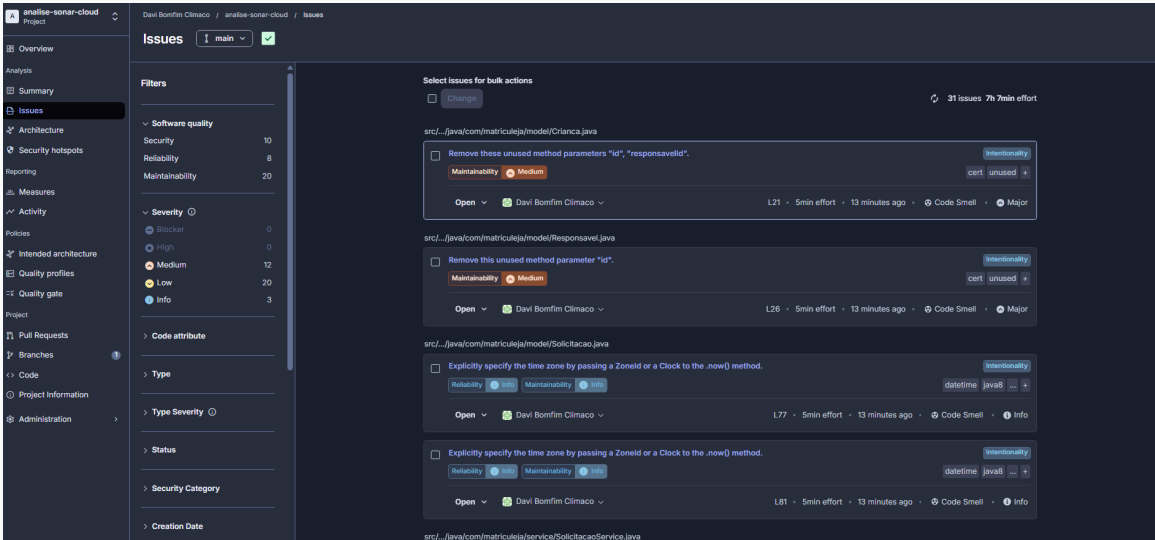
3. Resultados da Análise

Abaixo estão os principais resultados obtidos após a execução da análise no SonarCloud. Os números devem ser atualizados conforme o print gerado pela ferramenta.

Bugs	Vulnerabilidades	Code Smells	Cobertura de Testes
8	10	20	nao configurado



Print do painel principal do SonarCloud com os resultados da análise (bugs, code smells, coverage)



Print da lista de Code Smells identificados no projeto

4. Interpretação dos Resultados

4.1 O que encontramos

Com base na análise realizada pelo SonarQube, foram identificadas 38 ocorrências distribuídas entre segurança, confiabilidade e manutenção do código.

Os principais problemas encontrados foram:

- Parâmetros declarados e não utilizados em alguns métodos das entidades;
- Utilização de chamadas de data e hora sem especificação explícita de fuso horário;
- Alguns problemas de legibilidade e padronização apontados como Code Smells;
- Pequenas recomendações de melhoria relacionadas à manutenção futura do sistema.

4.2 Por que essas ocorrências apareceram

Grande parte das ocorrências identificadas está relacionada à qualidade e manutenção do código, e não a falhas de funcionamento do sistema.

Durante o desenvolvimento, o foco principal da equipe foi implementar corretamente as funcionalidades do MatriculeJá. Como consequência, algumas boas práticas de limpeza de código, padronização e refinamento ficaram para uma etapa posterior.

4.3 Qual o impacto real

Os problemas encontrados são classificados como de baixa severidade. Nenhum bug crítico ou falha de segurança foi identificado. O sistema funciona corretamente — os endpoints respondem como esperado, a lógica de prioridade está correta e os dados são manipulados de forma segura.

O SonarQube classificou o projeto com:

- Maintainability: A
- Security Rating: B
- Reliability Rating: B
- Quality Gate: Passed

Isso indica que o sistema atende aos critérios mínimos de qualidade definidos pela ferramenta e não apresenta falhas graves que comprometam seu funcionamento.

Além disso, o índice de duplicação de código foi reduzido para apenas **0,5%**, indicando boa reutilização e organização das classes.

4.4 O que faríamos para melhorar

- Remover parâmetros não utilizados;
- Ajustar o tratamento de datas utilizando fuso horário explícito;
- Expandir a documentação do código;
- Implementar testes automatizados e integrar cobertura de testes ao SonarQube;
- Corrigir gradualmente os Code Smells restantes para aumentar a manutenibilidade.

5. Conclusão

A análise realizada mostrou que o MatriculeJá apresenta boa qualidade estrutural, obtendo aprovação no Quality Gate do SonarQube e baixos índices de duplicação de código. As

ocorrências encontradas estão relacionadas principalmente à manutenção e boas práticas de desenvolvimento, não comprometendo o funcionamento do sistema. A utilização da ferramenta permitiu identificar oportunidades de melhoria e reforçou a importância da análise contínua da qualidade de software.

A análise foi um exercício muito útil para a equipe, porque ajudou a enxergar pontos que passam despercebidos durante o desenvolvimento. Num projeto real, rodar essa análise regularmente seria essencial para manter a qualidade do código ao longo do tempo.